

Jackson Horan

Professor Irakliotis

COMP 363: Algorithms

24 September 2026

Classic vs. Karatsuba Multiplication: A Timing Study

Both algorithms fit $T(n) = rT(n/c) + n^d$ with $c = 2$, since `split_in_half` halves each factor. Classic makes four recursive calls ($r = 4$); Karatsuba makes three ($r = 3$). All other work is linear ($d = 1$): `split_in_half`, `shift`, `add_digits`, and `subtract_digits` each pass once over at most n digits, a fixed number of times per call. Since $r > c^d = 2$, the Master Theorem predicts $O(n^2)$ for classic and $O(n^{\log_2 3}) \approx O(n^{1.585})$ for Karatsuba (Irakliotis, “Recurrence”).

Counting only digit multiplications, $M(n) = 4M(n/2)$ and $K(n) = 3K(n/2)$ with $M(1) = K(1) = 1$, so $M(n) = 4^{\log_2 n} = n^2$ and $K(n) = 3^{\log_2 n} = n^{\log_2 3}$. My counts matched exactly: $262,144 = 512^2$ and $19,683 = 3^9$ at $n = 512$.

Classic was faster through $n = 16$; Karatsuba won from $n = 32$ on, taking 77.5 ms vs. 281.6 ms at $n = 512$. On small inputs its extra additions and list copies outweigh the one multiplication it saves. Doubling n multiplied classic’s time by 4.0–4.1 and Karatsuba’s by 3.0–3.1, as $\Theta(n^2)$ and $\Theta(n^{\log_2 3})$ predict.

At $n = 512$, Karatsuba did 13.3× fewer multiplications but ran only 3.6× faster, because most of the time goes to linear list work, and Karatsuba does more of it per call. That gap held at about 3.6 from $n = 32$ on: a constant factor, not a different growth rate.

For $n \geq 8$, medians stayed within 12% of the minimum; tiny inputs varied up to 2.3×. Noise only adds time, so the minimum of seven trials is the most repeatable estimate.

Works Cited

Irakliotis, Leo. “Assignment: Classic vs. Karatsuba Multiplication—A Timing Study.” *COMP 363: Algorithms*, Fall 2026.

---. “Recurrence Analysis.” *COMP 363: Algorithms*, Fall 2026, `master_theorem.ipynb`.